

**CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY**

COLLEGE OF COMPUTER STUDIES

Software Design Description
for
PRISM

Change History Signature

Version	Date	Author	Description	Signature
1.0	5/12/2026	Pangilinan, Vein Carmell	Initial draft for SDD	
1.1	5/13/2026	Pangilinan, Vein Carmell	Added Class Diagram and Sequence Diagram for Module 2 and 3	
1.2	5/13/2026	Atamosa, Charry Mae	Added Class Diagram and Sequence Diagram for Module 1	
1.3	5/14/2026	Beato, Angel Anel	Added Class Diagram and Sequence Diagram for Module 4	
2.0	5/21/2026	Binagatan, Alexander Jr	Major architectural refinement of system design and technology stack sections.	

Preface

This Software Design Description (SDD) is prepared by the PRISM development team at Cebu Institute of Technology – University (CIT-U), College of Computer Studies, for Academic Year 2025–2026.

The document provides the complete technical design of PRISM (Parenteral Return-demonstration Injection Skills Monitor) — a Flutter Android application that uses MediaPipe computer vision, Firebase cloud services, and the OpenRouter API (Llama 3.3 70B Instruct) to assess nursing students' parenteral injection technique in real time.

This SDD is intended for the development team and serves as the primary technical reference throughout implementation. Readers are expected to have familiarity with Flutter/Dart, object-oriented design, and mobile application architecture.

The document is organized as follows:

- **Section 1 – Introduction** defines the purpose, scope, acronyms, and references.
- **Section 2 – Architectural Design** describes the system's layered architecture and technology stack.
- **Section 3 – Detailed Design** covers the four modules: Detection Engine, AI Feedback Engine, Role-based Access and Student View, and Instructor Dashboard.

Table of Contents

1.	INTRODUCTION.....	5
1.1.	PURPOSE.....	5
1.2.	SCOPE.....	5
1.3.	DEFINITIONS AND ACRONYMS.....	5
1.4.	REFERENCES.....	7
2.	ARCHITECTURAL DESIGN.....	8
3.	DETAILED DESIGN.....	11
	<i>Module 1.....</i>	<i>11</i>
	<i>Module 2.....</i>	<i>20</i>
	<i>Module 3.....</i>	<i>23</i>
	<i>Module 4.....</i>	<i>31</i>

1. Introduction

1.1. Purpose

This Software Design Description (SDD) document describes the internal design of PRISM — the Parenteral Return-demonstration Injection Skills Monitor. It translates the functional and non-functional requirements defined in the SRS into concrete architectural decisions, component designs, class structures, sequence flows, and data schemas that will guide the development team in implementing the system.

This document is intended for the PRISM development team at CIT-U's College of Computer Studies and serves as the primary technical reference throughout the second semester of Academic Year 2025–2026.

1.2. Scope

This SDD covers the complete technical design of PRISM across four modules:

- Module 1: Detection Engine — real-time MediaPipe-based computer vision pipeline for insertion angle, aspiration, and withdrawal angle detection
- Module 2: AI Feedback Engine — OpenRouter API integration with Llama 3.3 70B Instruct for clinical feedback generation
- Module 3: Role-based Access and Student View — Firebase Authentication, injection type selection, and student session results interface
- Module 4: Instructor Dashboard and Feedback Review — instructor-facing dashboard, feedback review, and controlled release interface

The design is implemented as a Flutter Android application using Dart, with MediaPipe for on-device inference, Firebase for cloud services, and the OpenRouter API for AI integration.

1.3. Definitions and Acronyms

Term / Acronym	Definition
PRISM	Official 1-word product name. Parenteral Return-demonstration Injection Skills Monitor.
Flutter	Google's open-source UI toolkit used to build the Android mobile application.
Dart	Programming language used by Flutter for application logic.
MediaPipe	Google's on-device ML framework for real-time hand landmark detection.
Hand Landmarker	MediaPipe solution providing 21 hand landmarks per camera frame.

Landmark 0	Wrist landmark — base point for vector-based angle computation.
Landmark 8	Index fingertip landmark — distal point for angle vector computation.
Landmark 4	Thumb tip landmark — used for aspiration plunger withdrawal tracking.
Llama 3.3	Meta's 70B Instruct LLM accessed via OpenRouter API for clinical feedback.
OpenRouter API	REST API gateway used to access Llama 3.3 for AI feedback generation.
Firebase Auth	Firebase Authentication — cloud user identity and role management service.
Firestore	Firebase Firestore — NoSQL cloud database for session record storage.
Flutter Provider	State management package used for app-wide state sharing.
MAE	Mean Absolute Error — angle detection accuracy metric (target: $\leq 3^\circ$).
UC	Use Case — as defined in the SRS document.
IM	Intramuscular injection — target angle 90° .
SubQ	Subcutaneous injection — target angle 45° .
IV	Intravenous injection — target angle 15° .
ID	Intradermal injection — target angle 10° .
API	Application Programming Interface.
JSON	JavaScript Object Notation — data interchange format used in API calls.
REST	Representational State Transfer — architectural style for API communication.
HTTPS	Hypertext Transfer Protocol Secure — encrypted communication protocol.

1.4. References

- Google. (2024). MediaPipe Hand Landmarker API. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- Google. (2024). Flutter Documentation. <https://docs.flutter.dev>
- Google. (2024). Firebase Authentication Documentation. <https://firebase.google.com/docs/auth>
- Google. (2024). Firebase Firestore Documentation. <https://firebase.google.com/docs/firestore>
- OpenRouter. (2024). OpenRouter API Documentation. <https://openrouter.ai/docs>
- Meta. (2024). Llama 3.3 70B Instruct. <https://ai.meta.com/blog/meta-llama-3/>
- Flutter Provider Package. <https://pub.dev/packages/provider>
- Walsh, C., et al. (2021). Rater agreement and the assessment of injection angle in nursing simulation. *Nurse Education Today*, 98, 104723.

2. Architectural Design

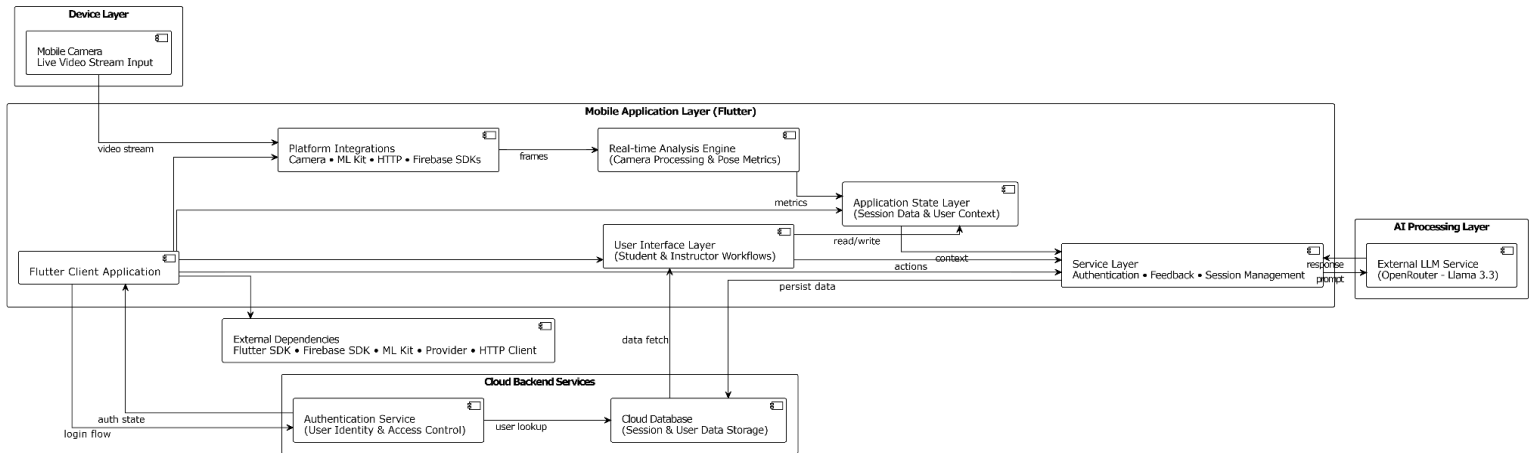
PRISM implements a layered client-service architecture: a Flutter Android application provides presentation and runtime business logic and performs on-device pose inference, while cloud services handle identity, persistent session storage, and LLM-based feedback generation. On-device MediaPipe/ML Kit pose inference runs independently of network connectivity; AI feedback is produced by the OpenRouter (Llama 3.3) API.

2.1. Architectural Overview

- **Mobile App (Flutter)** – The system uses a single Flutter application that initializes Firebase, registers providers, and manages the app's UI, detection workflow, and service integrations.
- **UI Screens & Navigation** – The application interface consists of student and instructor screens that handle navigation, user interaction, and business logic execution through providers and predefined workflows.
- **Real-time Detection Module** – The detection module captures camera input, performs on-device pose detection, computes injection-related metrics, and updates live session data in real time.
- **State Management (Providers)** – Providers manage and synchronize session data and user role information across the application as the central source of state.
- **Client Services & Orchestration** – Service classes coordinate authentication, AI feedback generation, and Firestore database operations for the application.
- **Authentication & Role Routing** – Firebase Authentication verifies users and routes them to student or instructor interfaces based on their assigned Firestore role.
- **Persistence & Data Model (Firestore)** – Cloud Firestore stores persistent application data such as user information, session metrics, scores, and AI-generated feedback.
- **LLM / External AI API** – The system generates AI-based clinical feedback by sending session metrics to an external OpenRouter/Llama 3.3 language model API.
- **Flutter Plugins & Libraries** – The application relies on Flutter plugins and libraries for camera access, pose detection, state management, Firebase integration, networking, and formatting utilities.
- **Data Flow & Privacy Constraints** – The system processes camera data into computed metrics and feedback while ensuring that session videos are not stored for privacy protection.

2.2. System Architecture Block Diagram

PRISM - System Architecture Block Diagram



The block diagram above shows the following components and their relationships:

1. Mobile App (Flutter)

The system is built as a single cross-platform Flutter application that initializes Firebase services, sets up global state management, and serves as the main runtime interface for all user interactions. It handles camera input, manages application state, coordinates service operations, and controls navigation between different parts of the system.

2. UI Screens & Navigation

The application consists of distinct student and instructor workflows that manage different user experiences. These interfaces handle real-time interaction, session progression, detection triggering, and feedback-related actions, ensuring a structured flow between live operation and session completion.

3. Real-time Detection Module

A real-time processing system analyzes live camera input using on-device pose detection technologies such as Google ML Kit and MediaPipe. It computes key performance metrics including movement angles, aspiration-related indicators, motion smoothness, and scoring outputs, which are continuously updated during active sessions.

4. State Management (Providers)

A centralized state management system maintains session-related data and user identity information throughout the application lifecycle. This ensures consistent access to live metrics, computed scores, and user role information across all components during runtime.

5. Client Services & Orchestration

Service components manage the core application logic, including authentication handling, session

processing, AI feedback generation, and session lifecycle operations. These services coordinate interactions between the user interface, state system, and external systems to ensure smooth workflow execution.

6. Authentication & Role Routing

User authentication is handled through Firebase Authentication, which verifies identity and assigns access roles. Based on the authenticated role, the system dynamically directs users to the appropriate interface, enforcing role-based access control.

7. Persistence & Data Model (Cloud Firestore)

The system uses Cloud Firestore as its primary database for storing persistent data, including user profiles, session records, performance metrics, AI-generated feedback, and instructor annotations. Each session record contains structured evaluation data used for analysis and review.

8. LLM / External AI API (OpenRouter → Llama 3.3)

AI-generated feedback is produced through an external large language model accessed via OpenRouter and powered by Llama 3.3. Session metrics are structured into prompts and sent to the model, which returns contextualized feedback stored alongside session data.

9. Device & Camera Hardware

The system relies on the device's physical camera to capture live video streams for real-time analysis. On-device processing is prioritized to reduce latency and enhance privacy, requiring actual mobile hardware for accurate operation.

10. Plugins & Libraries

The application utilizes Flutter-based plugins and supporting libraries to enable camera access, pose detection, authentication, state management, cloud integration, and external communication with backend services.

11. Data Flow & Privacy Constraints

The end-to-end workflow begins with live camera input, which is processed into measurable metrics through pose detection. These metrics are then used to generate AI-based feedback and stored for evaluation. For privacy protection, raw video data is not stored; only computed metrics and textual feedback are retained.

2.3. Technology Stack

Layer	Technology	Purpose
Mobile Framework	Flutter (Dart 3.11.5)	Cross-platform framework used to build the entire client application, including UI, state initialization, and runtime orchestration.
State Management	Provider (provider package)	Manages global application state, including live session metrics and user role information through centralized state controllers.
Computer Vision	Google ML Kit Pose Detection (MediaPipe landmarks)	Performs on-device real-time pose detection to extract landmarks used for computing movement angles, aspiration metrics, and performance indicators.
AI / LLM	Llama 3.3 (70B) via OpenRouter API (HTTP REST)	Generates structured clinical feedback from session metrics through external API requests handled by a dedicated service layer, with support for mock mode during development.
Authentication	Firebase Authentication (firebase_auth)	Handles user authentication, identity management, and initial session routing, with role information retrieved from the database layer.
Database	Firebase Firestore (cloud_firestore)	Stores persistent application data including user profiles and session records, following a structured schema for consistent data handling.
Camera Access	Flutter camera plugin	Provides real-time video stream input from the device's rear camera for processing by the detection module.
HTTP Client	Dart http package	Handles secure REST API communication with external services such as OpenRouter for

		AI inference requests.
Target Platform	Android 8.0+ (API Level 26+)	Primary deployment environment optimized for real-time processing, camera access, and on-device inference.
Dev / Test Utilities	Dart environment configuration flags	Enables development modes such as mock AI responses (USE MOCK FEEDBACK=true) and controlled environment-based API configuration for testing and debugging.
Core Libraries & Plugins	Flutter ecosystem dependencies	Includes essential packages for Firebase integration, pose detection, state management, camera handling, networking, authentication, and localization support.

3. Detailed Design

Module 1: Detection Engine

Corresponds to General Objective 1. Implements real-time MediaPipe-based computer vision for needle insertion angle (UC-1.1), aspiration technique (UC-1.2), and withdrawal angle detection (UC-1.3).

1.1 Real-time Needle Insertion Angle Detection (UC-1.1)

User Interface Design

The detection screen presents a full-screen camera preview with real-time overlays. The angle value is displayed as a pill badge at the top center. Hand skeleton landmarks (Landmark 0 and Landmark 8) are rendered as dots connected by a line with an angle arc annotation. A bottom panel shows the current session phase status (Insertion, Aspiration, Withdrawal). A 'Detection Lost' warning banner appears when landmarks cannot be tracked.

Front-end Components

Component Name	DetectionScreen (Flutter StatefulWidget)
Description	Full-screen camera preview widget that hosts the CameraPreview, CustomPaint overlay for landmark rendering, angle display badge, phase status panel, and the Confirm Needle Insertion button.
Component Type	Flutter StatefulWidget / Screen

Component Name	AngleOverlayPainter (CustomPainter)
Description	CustomPainter that draws the hand landmark skeleton (Landmark 0 to Landmark 8), the angle arc, and the real-time angle value annotation directly onto the camera preview canvas.
Component Type	Flutter CustomPainter

Component Name	AngleBadgeWidget
Description	Displays the computed insertion angle value in a navy pill badge with a Live/Locked status indicator. Updates in real-time during detection and locks upon insertion confirmation.
Component Type	Flutter StatelessWidget

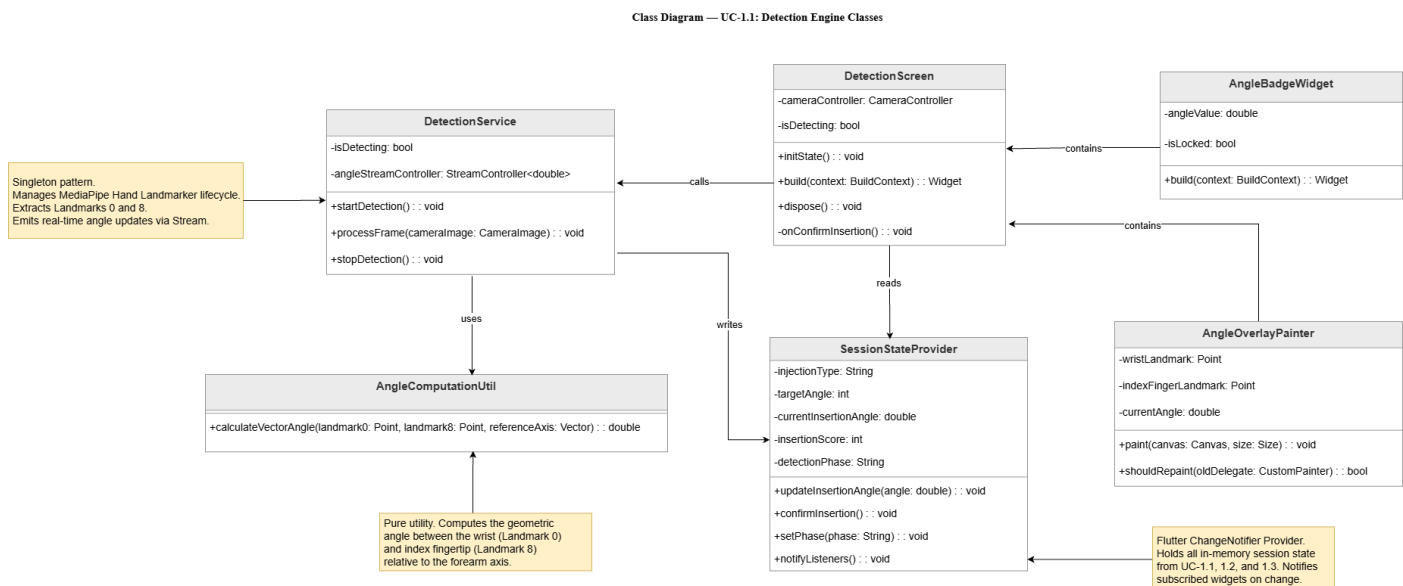
Back-end Components

Component Name	DetectionService (Dart class)
Description	Manages the MediaPipe Hand Landmarker lifecycle, processes each camera frame, extracts Landmarks 0 and 8, computes the insertion angle using dot product vector geometry relative to the forearm axis, and emits angle updates via a Stream.
Component Type	Dart Service Class / Singleton

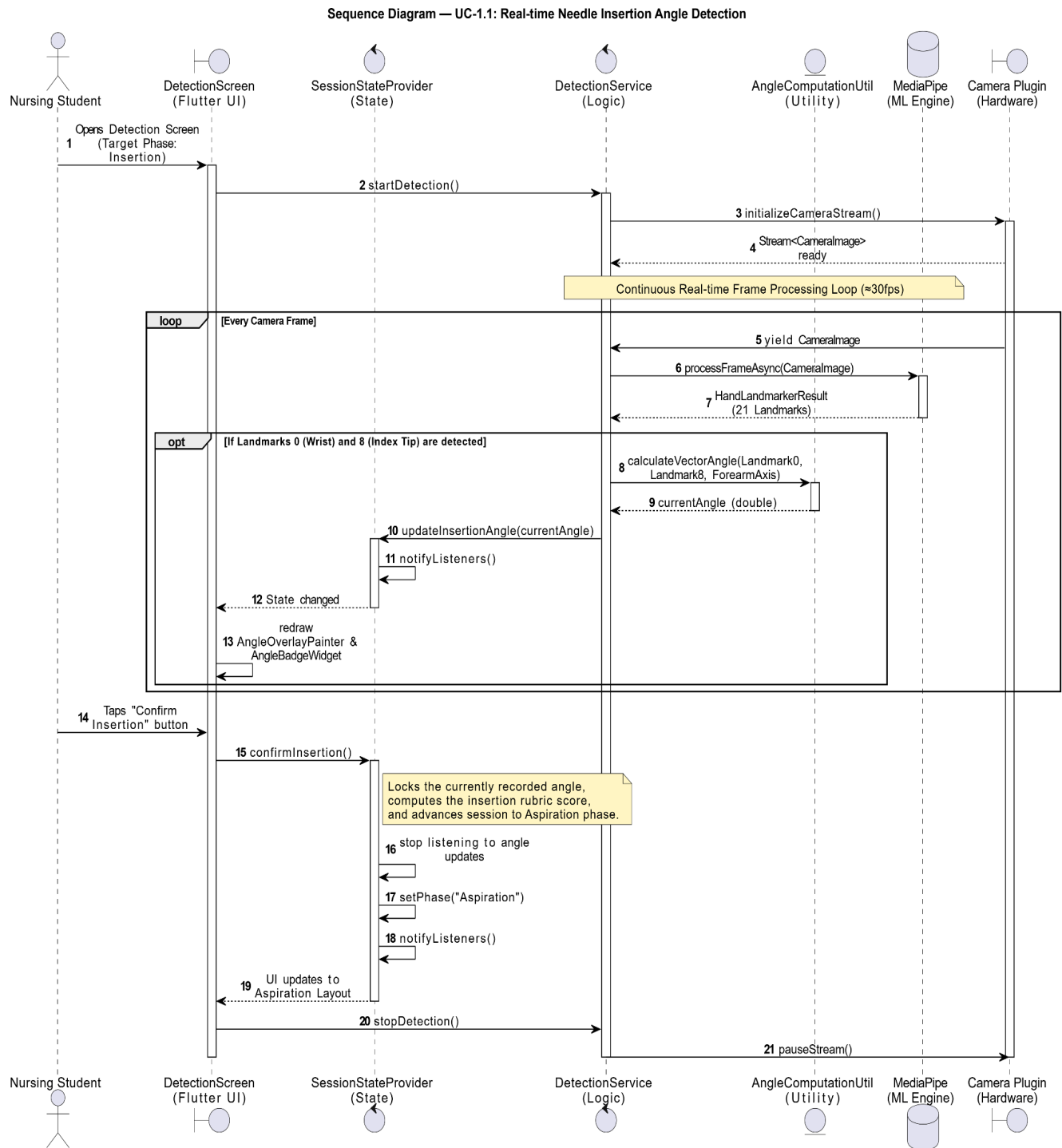
Component Name	AngleComputationUtil (Dart utility)
Description	Pure utility class implementing the vector-based geometric angle computation: calculates the angle between the wrist-to-fingertip vector (L0→L8) and the vertical/forearm reference axis using atan2. Returns angle in degrees.
Component Type	Dart Utility Class (static methods)

Component Name	SessionStateProvider (Provider)
Description	Flutter Provider that holds the current session state: selected injection type, target angle, detection phase, recorded insertion angle value, aspiration result, withdrawal angle value, and rubric scores. Notifies all subscribed widgets on state change.
Component Type	Flutter ChangeNotifier Provider

Object-Oriented Components Class Diagram



Sequence Diagram



Data Design

Session state for UC-1.1 is held in memory via SessionStateProvider and written to Firestore only upon session completion (UC-2.1). The relevant fields stored in the session document are:

Field	Type	Description
insertionAngle	double	Recorded insertion angle in degrees at confirmation
insertionScore	int	CIT-U five-point rubric score for insertion angle (1–5)
injectionType	String	Selected injection type: IM, SubQ, IV, or ID
targetAngle	int	Target angle for selected injection type (90, 45, 15, or 10)

1.2 Aspiration Technique Detection (UC-1.2)

User Interface Design

Following insertion confirmation, the detection screen transitions to the aspiration tracking state. The Landmark 4 (thumb tip) position is shown as a glowing dot overlay. A displacement progress indicator shows proximal-to-distal thumb movement. Motion control and duration metrics are displayed in the bottom status panel. A 'Not Detectable' amber warning banner appears when the thumb landmark is occluded.

Front-end Components

Component Name	AspirationOverlayPainter (CustomPainter)
Description	CustomPainter that renders the Landmark 4 (thumb tip) tracking dot, the displacement trajectory trail, and the plunger withdrawal progress bar on the camera preview canvas.
Component Type	Flutter CustomPainter

Component Name	AspirationStatusPanel
Description	Bottom panel widget displaying real-time aspiration metrics: displacement in mm, duration in seconds, and motion smoothness rating. Updates continuously during detection.
Component Type	Flutter StatelessWidget

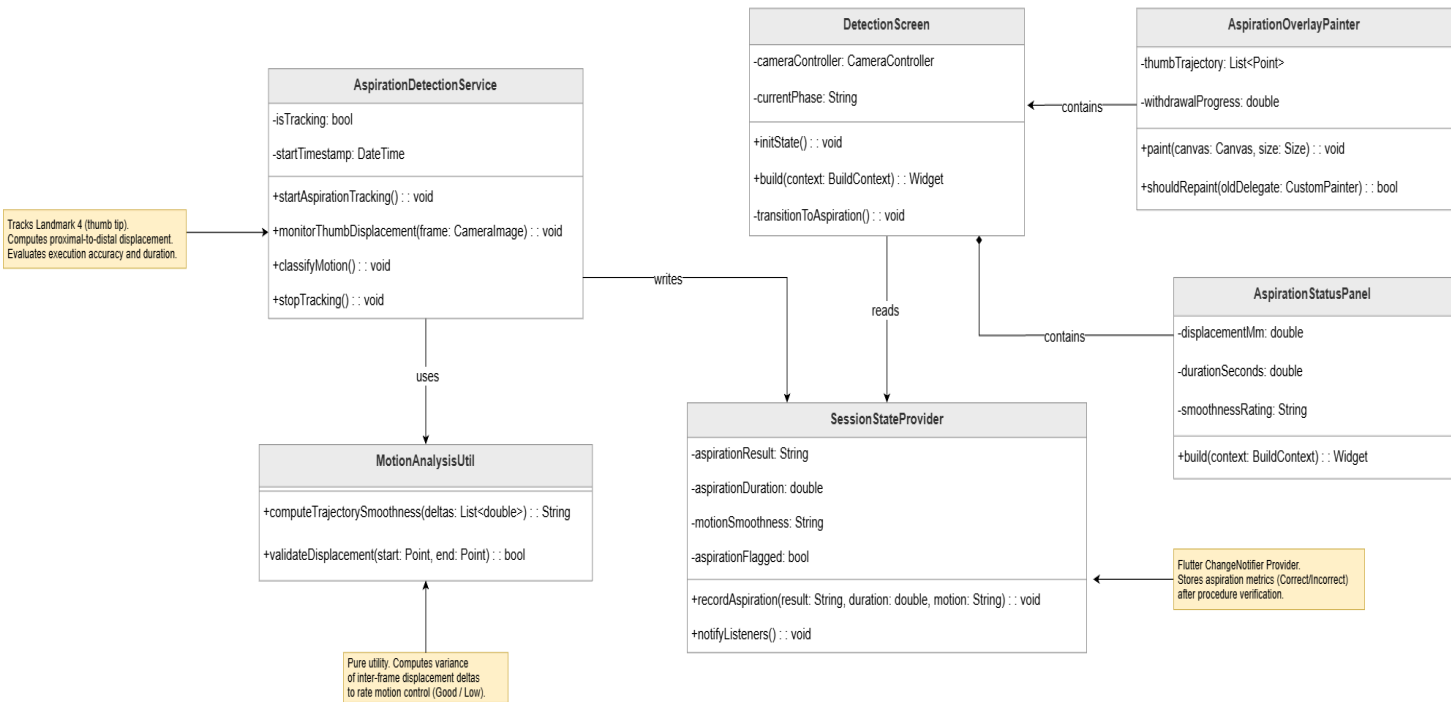
Back-end Components

Component Name	AspirationDetectionService (Dart class)
Description	Monitors Landmark 4 (thumb tip) displacement from the MediaPipe stream following insertion confirmation. Computes proximal-to-distal displacement, measures withdrawal duration, evaluates motion smoothness via trajectory variance analysis, and classifies the result as Correct, Incorrect, or Not Detected.
Component Type	Dart Service Class

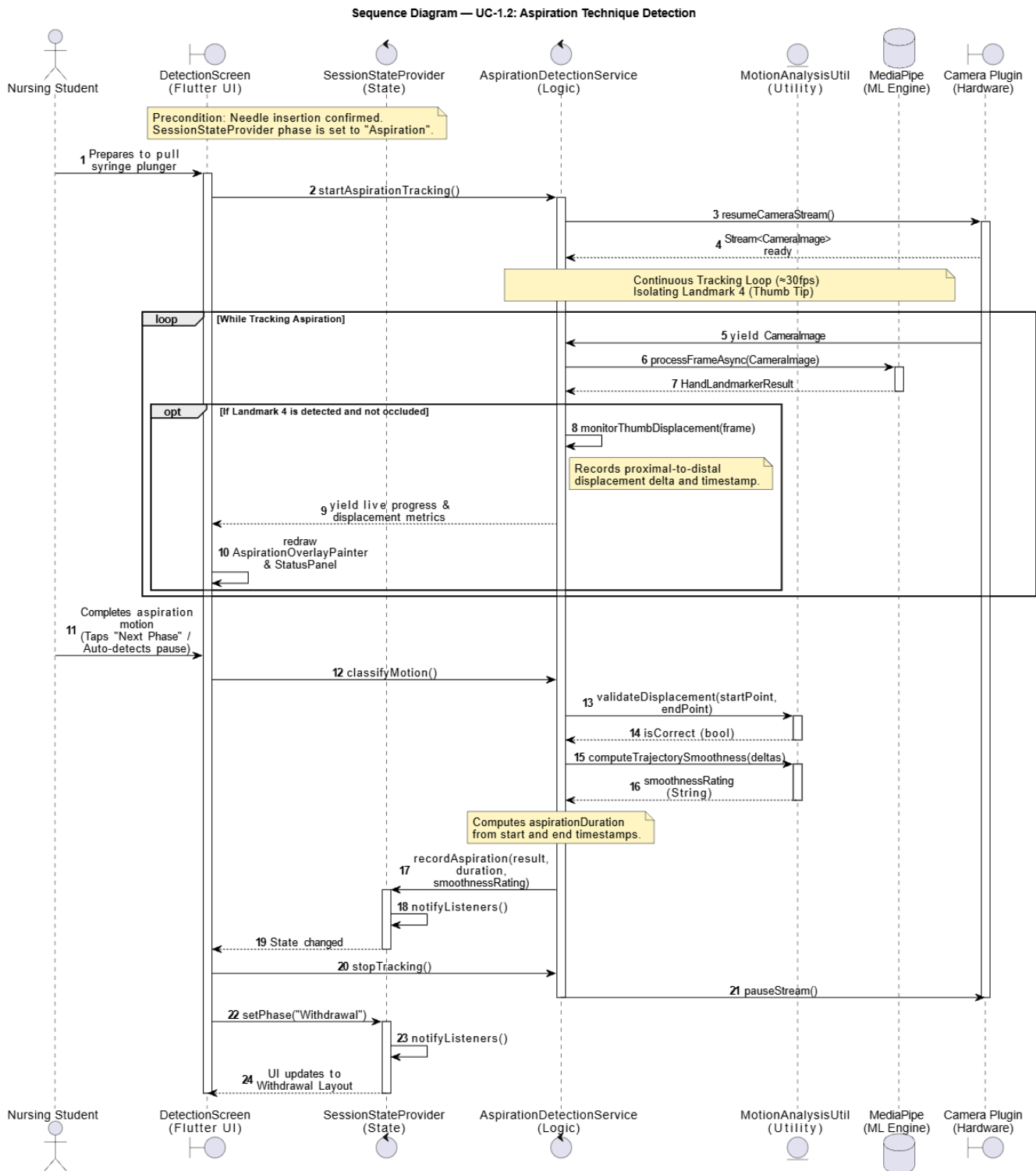
Component Name	MotionAnalysisUtil (Dart utility)
Description	Utility class that computes trajectory smoothness (variance of inter-frame displacement deltas) and validates minimum displacement threshold and duration requirements against CIT-U rubric standards.
Component Type	Dart Utility Class (static methods)

Object-Oriented Components
Class Diagram

Class Diagram — UC-1.2: Aspiration Detection Classes



Sequence Diagram



Data Design

Field	Type	Description
aspirationResult	String	Correct / Incorrect / Not Detected
aspirationDuration	double	Duration of plunger withdrawal in seconds
motionSmoothness	String	Good / Low — based on trajectory variance threshold
aspirationFlagged	bool	True if thumb landmark was occluded during detection

1.3 Needle Withdrawal Angle Detection and Correspondence Verification (UC-1.3)

User Interface Design

The detection screen transitions to the withdrawal tracking state after aspiration completes. The same vector overlay used in UC-1.1 is reactivated. A correspondence preview panel shows the stored insertion angle alongside the live withdrawal angle and computes the delta in real-time. Upon withdrawal confirmation, a Matches / Deviates result badge is displayed.

Front-end Components

Component Name	WithdrawalOverlayPainter (CustomPainter)
Description	Reuses the AngleOverlayPainter logic for withdrawal angle display. Additionally renders a side-by-side correspondence preview showing stored insertion angle vs. live withdrawal angle with delta annotation.
Component Type	Flutter CustomPainter

Component Name	CorrespondenceResultBadge
Description	Widget that displays the final correspondence result (Matches in green / Deviates in red) with the angular delta value and CIT-U rubric step reference upon withdrawal confirmation.
Component Type	Flutter StatelessWidget

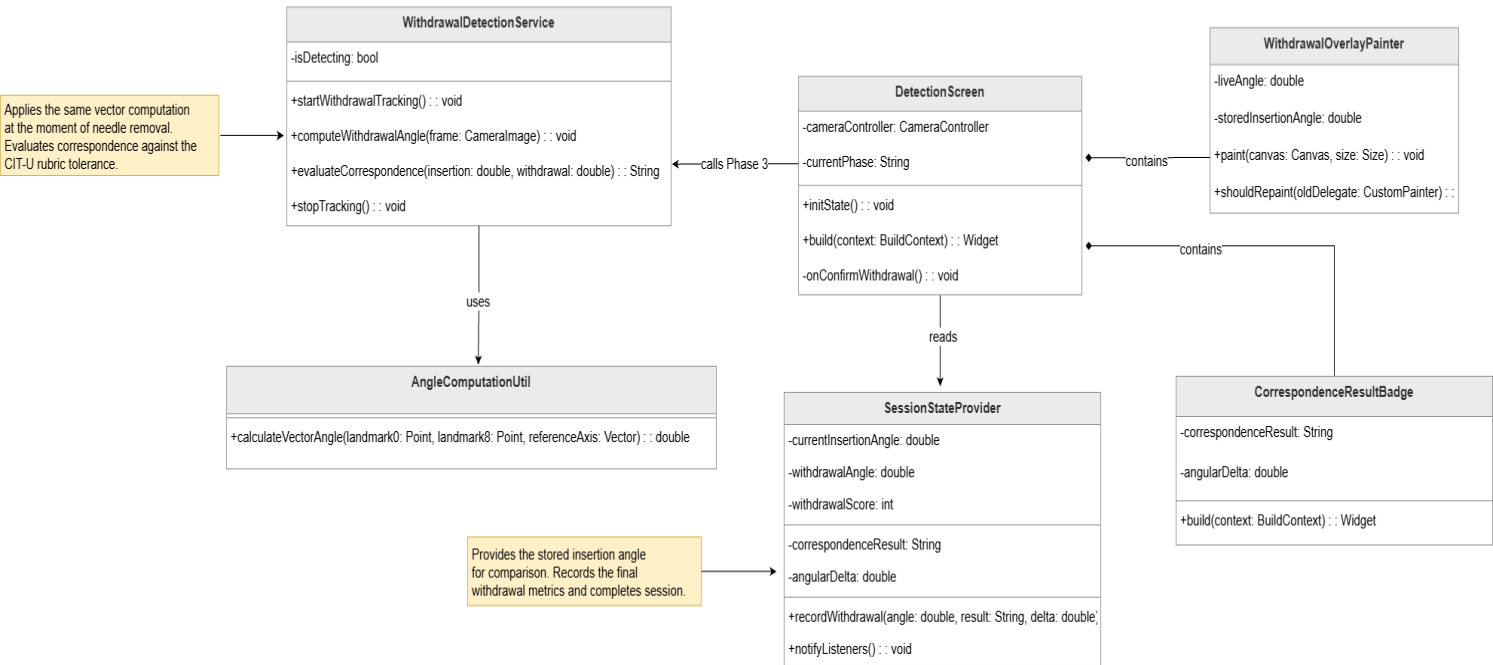
Back-end Components

Component Name	WithdrawalDetectionService (Dart class)
----------------	---

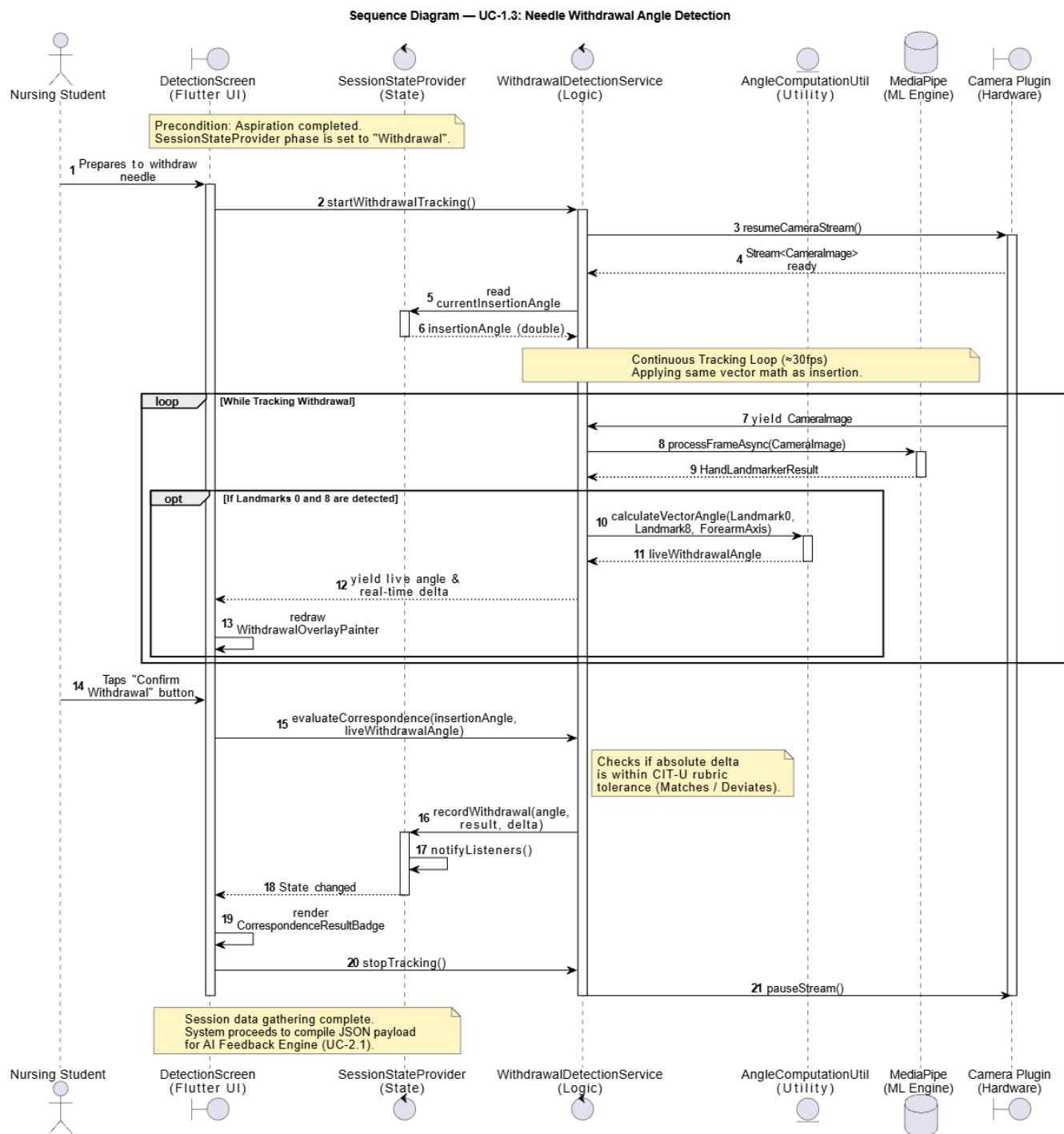
Description	Applies the same vector computation as DetectionService (Landmarks 0 and 8 relative to forearm axis) at the moment of needle withdrawal. Retrieves the stored insertion angle from SessionStateProvider, computes the angular delta, evaluates correspondence within CIT-U rubric tolerance, and scores against the five-point rubric (Step 22).
Component Type	Dart Service Class

Object-Oriented Components
Class Diagram

Class Diagram — UC-1.3: Needle Withdrawal Angle Detection



Sequence Diagram



Data Design

Field	Type	Description
withdrawalAngle	double	Recorded withdrawal angle in degrees
withdrawalScore	int	CIT-U five-point rubric score for withdrawal (1–5)
correspondenceResult	String	Matches / Deviates
angularDelta	double	Absolute difference between insertion and withdrawal angles
withdrawalFlagged	bool	True if withdrawal angle was Not Detectable

Module 2: AI Feedback Engine

Corresponds to General Objective 2. Compiles detection results from Module 1 and generates structured, clinically worded performance feedback using Llama 3.3 70B Instruct via the OpenRouter API (UC-2.1).

2.1 AI-Generated Clinical Feedback (UC-2.1)

User Interface Design

Upon session completion, the screen transitions to a Session Complete view. A loading state shows a Llama 3.3 processing indicator with a payload compilation summary (all three detection results shown as Ready). After feedback is generated, the screen shows a Session Recorded confirmation with a Pending Instructor Review status badge. If the API fails, an error state is shown with a Retry button and a note that raw results have been saved.

Front-end Components

Component Name	SessionCompleteScreen (Flutter StatefulWidget)
Description	Post-session screen displaying the feedback generation loading state, session summary card (three detection results), feedback pending confirmation, and navigation options (Return to Home, View My Sessions).

Component Type	Flutter StatefulWidget / Screen
Component Name	FeedbackLoadingWidget
Description	Animated loading indicator widget showing the Llama 3.3 via OpenRouter status, payload compilation progress, and estimated wait time (≈2 seconds).
Component Type	Flutter StatelessWidget

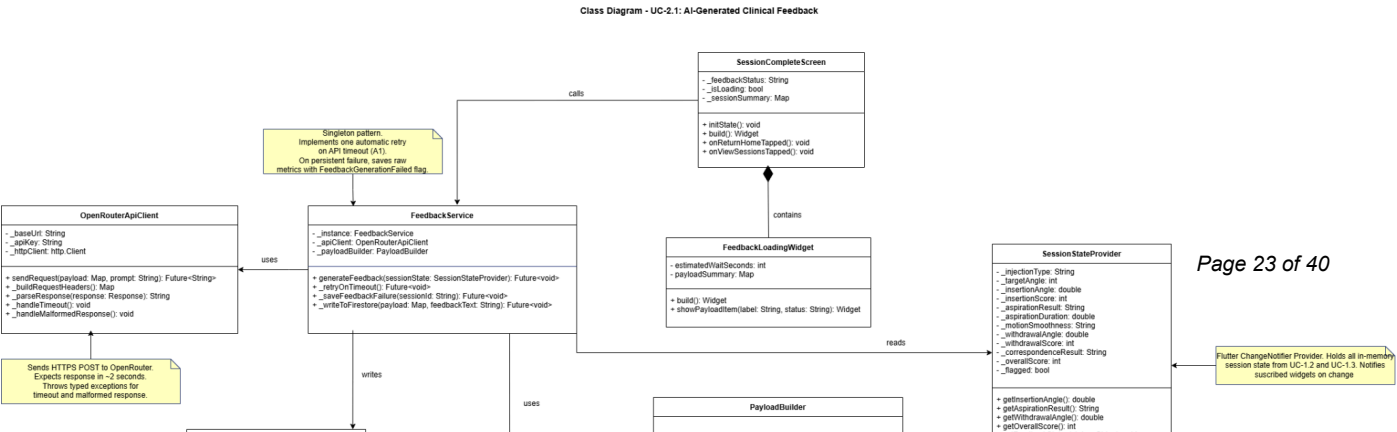
Back-end Components

Component Name	FeedbackService (Dart class)
Description	Orchestrates AI feedback generation. Compiles the session payload JSON from SessionStateProvider, sends it to the OpenRouter API endpoint via HTTPS POST, parses the Llama 3.3 response, and writes the result to Firestore with Pending status. Implements one automatic retry on timeout. On persistent failure, stores raw metrics with a Feedback Generation Failed flag.
Component Type	Dart Service Class / Singleton

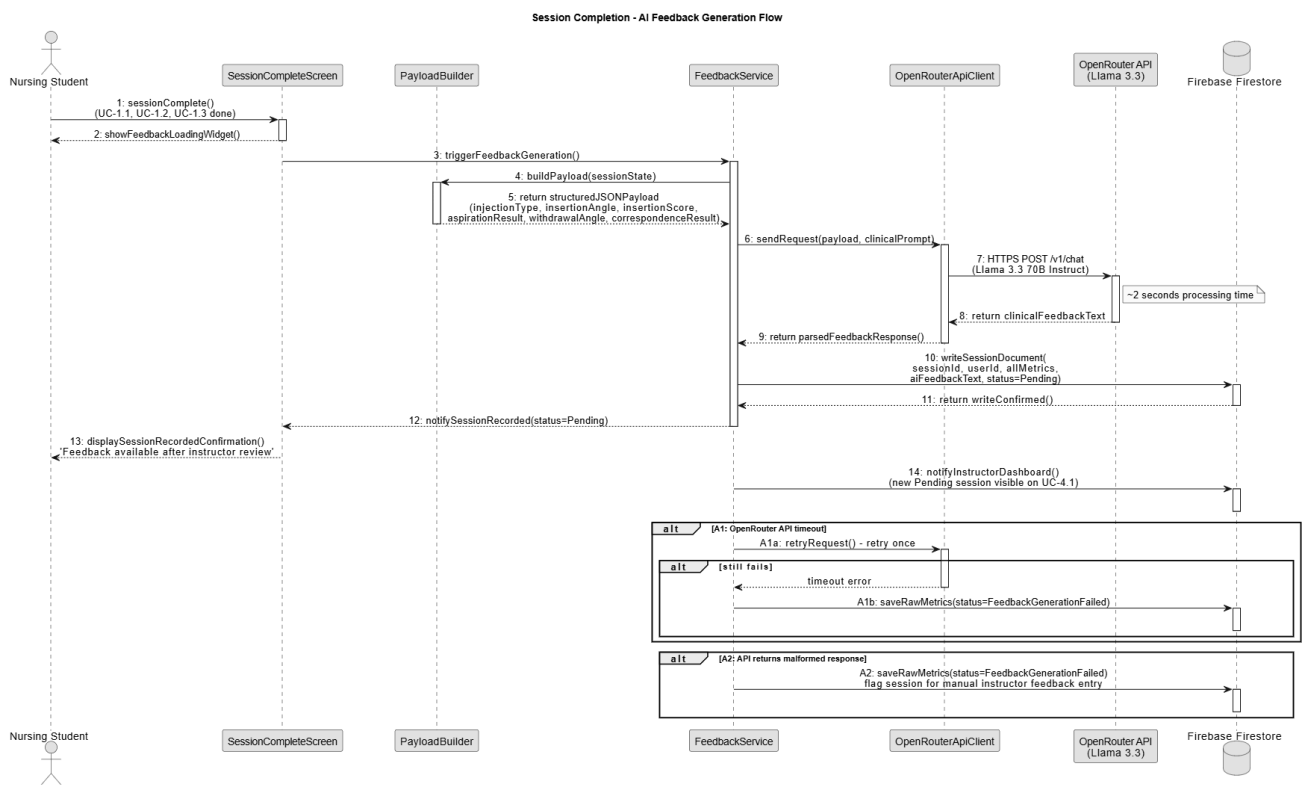
Component Name	OpenRouterApiClient (Dart class)
Description	HTTP client wrapper for the OpenRouter REST API. Constructs the HTTPS POST request with the clinical feedback generation prompt and detection data payload, handles response parsing, and throws typed exceptions for timeout and malformed response cases.
Component Type	Dart HTTP Client Class

Component Name	PayloadBuilder (Dart utility)
Description	Utility class that assembles the structured JSON payload from SessionStateProvider fields: injection type, insertion angle and score, aspiration result and metrics, withdrawal angle and correspondence result. Formats it with the CIT-U rubric-aligned clinical feedback prompt for Llama 3.3.
Component Type	Dart Utility Class (static methods)

Object-Oriented Components
Class Diagram



Sequence Diagram



Data Design

The Firestore session document schema used by Module 2:

Field	Type	Description
sessionId	String	Auto-generated Firestore document ID
userId	String	Firebase Auth UID of the student
timestamp	Timestamp	Server-side Firestore timestamp of session creation

injectionType	String	IM / SubQ / IV / ID
insertionAngle	double	Recorded insertion angle in degrees
insertionScore	int	CIT-U rubric score (1–5)
aspirationResult	String	Correct / Incorrect / Not Detected
aspirationDuration	double	Plunger withdrawal duration in seconds
motionSmoothness	String	Good / Low
withdrawalAngle	double	Recorded withdrawal angle in degrees
withdrawalScore	int	CIT-U rubric score (1–5)
correspondenceResult	String	Matches / Deviates
overallScore	int	Computed overall rubric score (1–5)
aiFeedbackText	String	Clinically worded feedback generated by Llama 3.3
feedbackStatus	String	Pending / Released / Feedback Generation Failed
releaseTimestamp	Timestamp	Server timestamp when instructor released feedback (null if Pending)
instructorNote	String	Optional instructor annotation added during review
flagged	bool	True if any detection component was Not Detectable

Module 3: Role-based Access and Student View

Corresponds to General Objective 3. Implements Firebase Authentication with role-based access control (UC-3.1), injection type selection and detection parameter configuration (UC-3.2), and the student session results interface (UC-3.3).

3.1 User Registration, Login, and Role-based Access (UC-3.1)

User Interface Design

Three screens comprise this transaction: the Login Screen (email/password fields, Login button, Register link), the Register Screen (full name, email, password, confirm password, role toggle between Nursing Student and Clinical Instructor, Register button), and the Email Verification Screen (envelope icon, verification status message, I Have Verified button, Resend link). All screens use the navy/white PRISM design system.

Front-end Components

Component Name	LoginScreen (Flutter StatefulWidget)
-----------------------	--------------------------------------

Description	Login screen with email and password TextFormFields with validation, a Login ElevatedButton, a Register TextButton link, and a connectivity error banner for the A3 alternative flow.
Component Type	Flutter StatefulWidget / Screen

Component Name	RegisterScreen (Flutter StatefulWidget)
Description	Registration screen with full name, email, password, and confirm password TextFormFields. Includes a role selection toggle (Nursing Student / Clinical Instructor) implemented as a ToggleButtons widget. Submits to Firebase Auth on Register tap.
Component Type	Flutter StatefulWidget / Screen

Component Name	EmailVerificationScreen (Flutter StatefulWidget)
Description	Verification pending screen that polls Firebase Auth for email verification status on a timer. Shows verification instructions and the user's email address. Provides Resend Verification Email and I Have Verified My Email buttons.
Component Type	Flutter StatefulWidget / Screen

Component Name	AuthWrapper (Flutter StatelessWidget)
Description	Root-level widget that listens to the Firebase Auth state stream. Routes authenticated users to their role-specific home screen (StudentHomeScreen or InstructorDashboardScreen) and unauthenticated users to LoginScreen.
Component Type	Flutter StatelessWidget / Router

Back-end Components

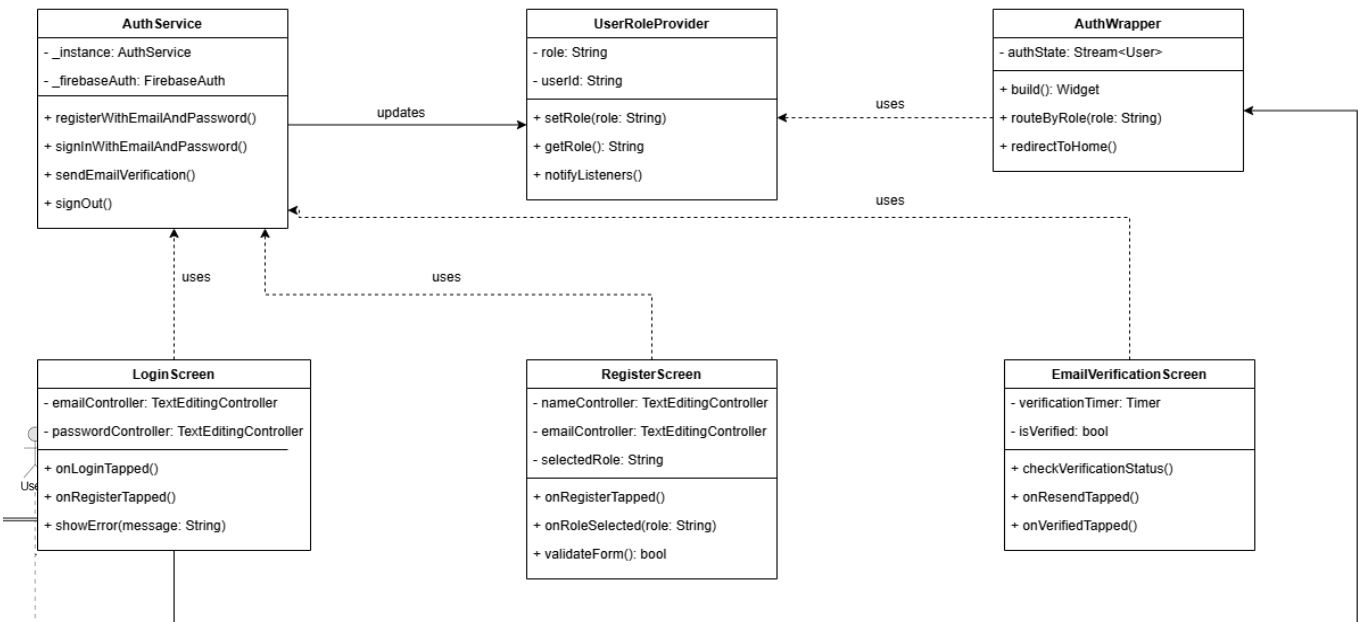
Component Name	AuthService (Dart class)
Description	Wraps Firebase Authentication SDK calls. Provides methods: registerWithEmailAndPassword(email, password, role, name), signInWithEmailAndPassword(email, password), sendEmailVerification(), signOut(), and a stream of Firebase User objects for AuthWrapper. Assigns role custom claims via Firestore user document on registration.
Component Type	Dart Service Class / Singleton

Component Name	UserRoleProvider (Provider)
Description	Flutter Provider that stores the authenticated user's role (Student or Instructor) read from the Firestore user document after login. Consumed by AuthWrapper and throughout the app for role-gated UI rendering.

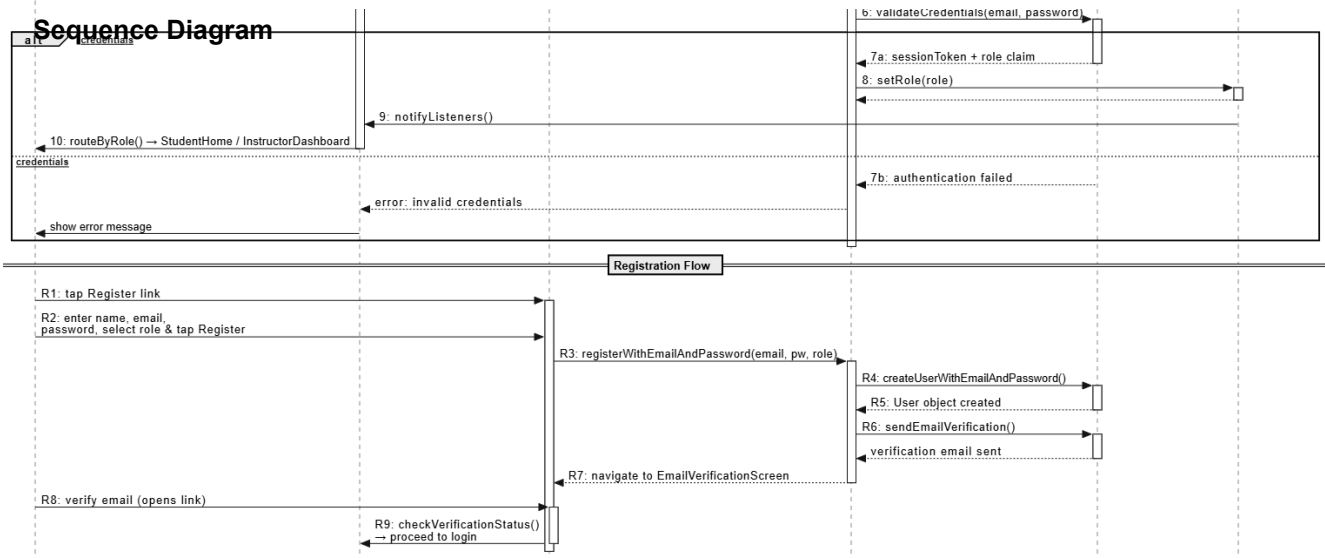
Component Type	Flutter ChangeNotifier Provider
----------------	---------------------------------

Object-Oriented Components
Class Diagram

Class Diagram - UC-3.1: User Registration, Login, and Role-based Access



Sequence Diagram



Data Design

Firestore 'users' collection document schema:

Field	Type	Description
uid	String	Firebase Auth UID — document ID
fullName	String	User's full name entered at registration
email	String	User's email address
role	String	Student or Instructor — assigned at registration
createdAt	Timestamp	Server timestamp of account creation
emailVerified	bool	Mirrored from Firebase Auth for Firestore queries

3.2 Injection Type Selection (UC-3.2)

User Interface Design

The Student Home Screen displays a welcome greeting, an overall progress stats card (avg score, total sessions, released/pending counts), a prominent Start New Session button, and a Recent Sessions list. The Injection Type Selection Screen shows four selectable cards (IM 90°, SubQ 45°, IV 15°, ID 10°) with injection name, description, site, and target angle. A Confirm Selection button activates when a card is selected.

Front-end Components

Component Name	StudentHomeScreen (Flutter StatefulWidget)
Description	Student-facing home screen showing progress stats, Start New Session ElevatedButton, and a scrollable list of recent session summary cards retrieved from Firestore. Tapping Start New Session navigates to InjectionTypeSelectionScreen.
Component Type	Flutter StatefulWidget / Screen

Component Name	InjectionTypeSelectionScreen (Flutter StatefulWidget)
----------------	---

Description	Full-screen selection screen displaying four InjectionTypeCard widgets. Tracks the selected injection type in local state. The Confirm Selection button triggers detection parameter configuration and navigates to DetectionScreen.
Component Type	Flutter StatefulWidget / Screen

Component Name	InjectionTypeCard (Flutter StatelessWidget)
Description	Selectable card widget displaying injection type name, badge label (IM/SubQ/IV/ID), clinical description, injection site, and target angle. Changes to navy filled style when selected.
Component Type	Flutter StatelessWidget

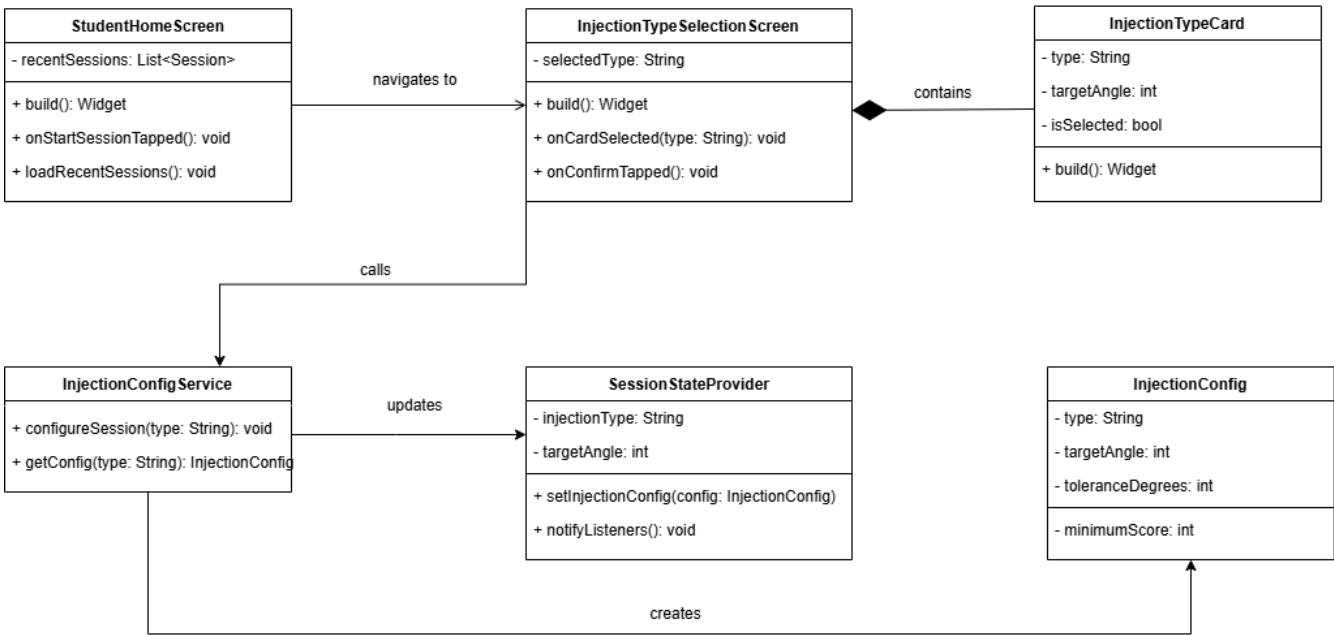
Back-end Components

Component Name	InjectionConfigService (Dart class)
Description	Configures SessionStateProvider with the detection parameters for the selected injection type: target angle (IM:90, SubQ:45, IV:15, ID:10), rubric success thresholds, and scoring weights. Called when the student confirms their injection type selection.
Component Type	Dart Service Class

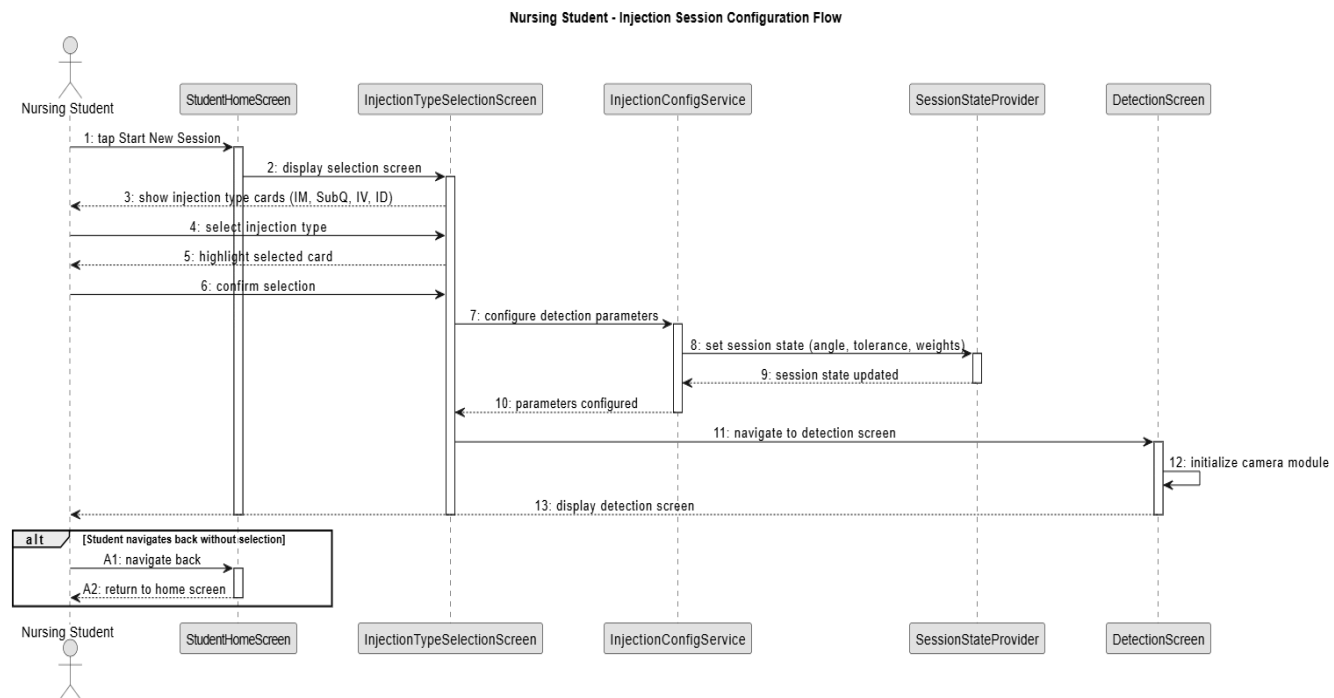
Object-Oriented Components

Class Diagram

Class Diagram - UC-3.2: Injection Type Selection



Sequence Diagram



Data Design

InjectionConfig is an in-memory data class (not persisted). The injection type value is stored in the Firestore session document as injectionType (String).

InjectionConfig Field	Type	Description
type	String	IM / SubQ / IV / ID
targetAngle	int	Target insertion angle in degrees
toleranceDegrees	int	CIT-U rubric tolerance for angle scoring
minimumScore	int	Minimum rubric score threshold (1–5)

3.3 Student Session Results View (UC-3.3)

User Interface Design

The My Sessions screen displays a session list with filter tabs (All, Released, IM, SubQ, IV, ID), session summary cards showing injection type, date, score, and status badge. Tapping a session navigates to the Session Detail screen showing all four detection result sections (Insertion Angle, Aspiration, Withdrawal Angle, Overall Score) and the AI Clinical Feedback panel with improvement guidance.

Front-end Components

Component Name	MySessionsScreen (Flutter StatefulWidget)
Description	Scrollable sessions list screen with filter chip tabs. Queries Firestore for Released sessions belonging to the authenticated student. Renders SessionSummaryCard widgets. Shows empty state message when no released sessions exist.
Component Type	Flutter StatefulWidget / Screen

Component Name	SessionDetailScreen (Flutter StatefulWidget)
Description	Detailed session results screen displaying detection result cards for all three components, overall rubric score, and the AI Clinical Feedback panel with the Llama 3.3 generated text and improvement guidance. Read-only for students.
Component Type	Flutter StatefulWidget / Screen

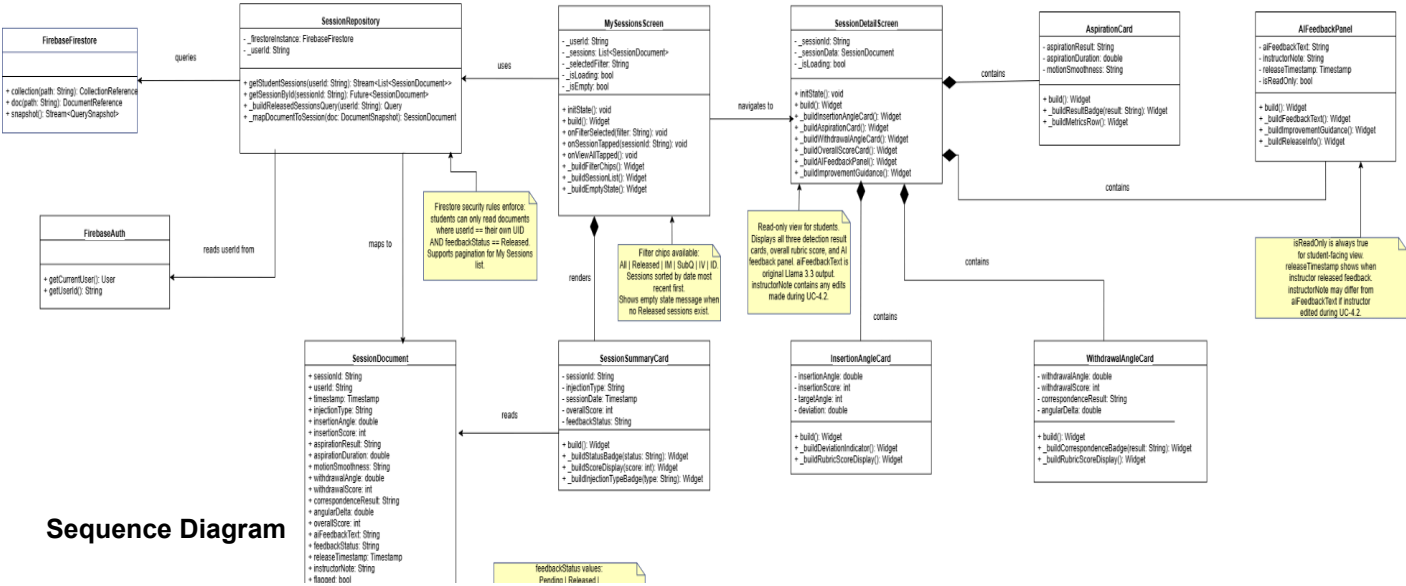
Back-end Components

Component Name	SessionRepository (Dart class)
----------------	--------------------------------

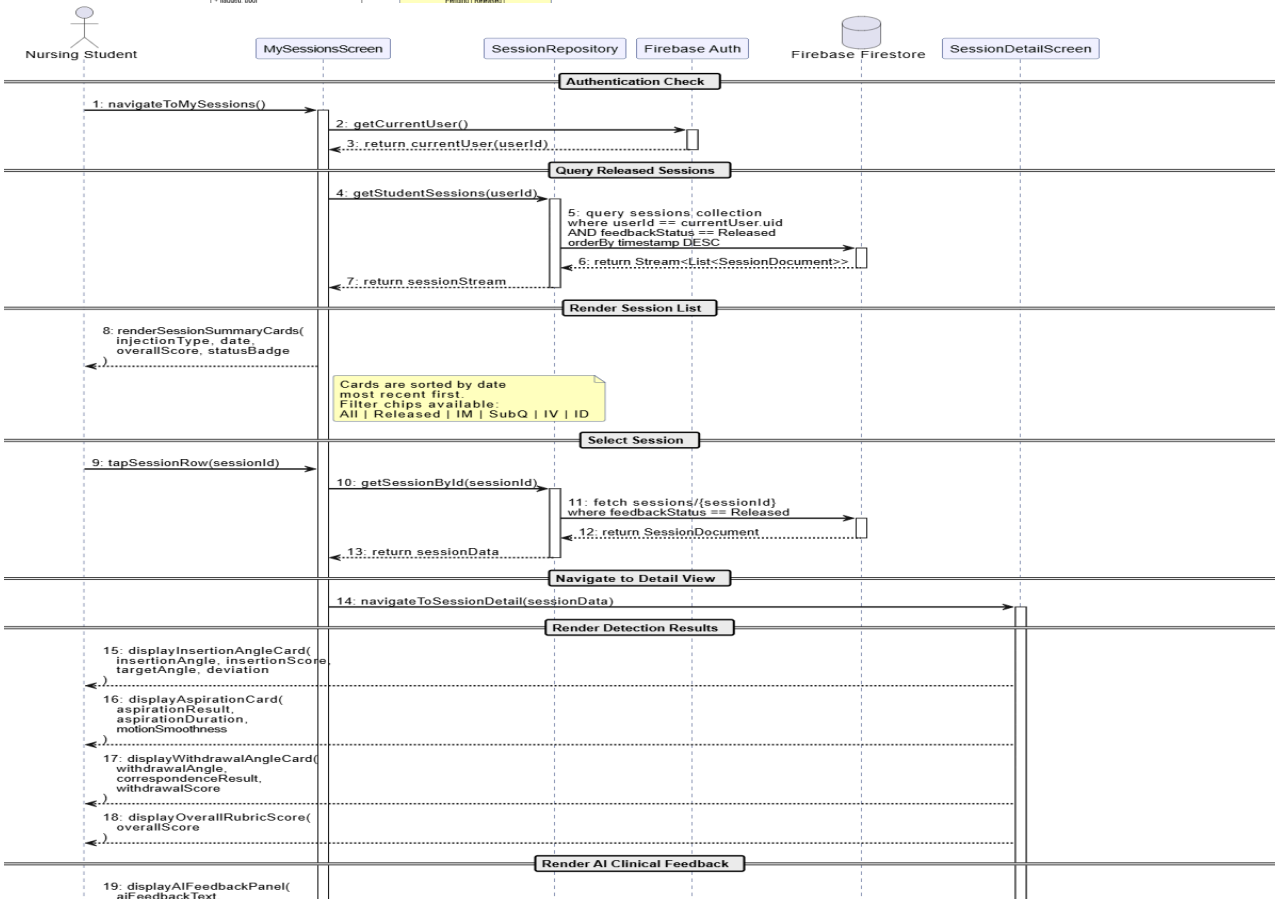
Description	Firestore data access layer for session records. Provides: getStudentSessions(userId) returning a Stream of Released session documents, getSessionById(sessionId) for detail view, and pagination support for the My Sessions list.
Component Type	Dart Repository Class

Object-Oriented Components
Class Diagram

Class Diagram - UC-3.3: Student Session Results View



Sequence Diagram



Data Design

Student session results are read from the Firestore 'sessions' collection (schema defined in Module 2, Section 2.1). Firestore security rules enforce that students can only read documents where `userId == their UID AND feedbackStatus == Released`.

Module 4: Instructor Dashboard and Feedback Review

Corresponds to General Objective 4. Implements the instructor-facing dashboard for monitoring student performance (UC-4.1) and the feedback review and controlled release interface (UC-4.2).

4.1 Instructor Performance Monitoring Dashboard (UC-4.1)

User Interface Design

The Instructor Dashboard shows a summary header (Total Sessions, Pending Review count, Released count), a search field, filter tabs (All, Pending, Released, injection types), and a scrollable session list table showing student name, injection type, session date, overall score, and status badge. Tapping a row navigates to the Feedback Review screen.

Front-end Components

Component Name	InstructorDashboardScreen (Flutter StatefulWidget)
Description	Instructor home screen displaying session summary stats, search and filter controls, and a scrollable SessionListTile list. Listens to a Firestore snapshot stream for real-time updates when new student sessions arrive. Shows empty dashboard message when no sessions exist.
Component Type	Flutter StatefulWidget / Screen

Component Name	SessionListTile (Flutter StatelessWidget)
Description	List tile widget showing student name, injection type badge, session date, overall score, and a colored status badge (amber for Pending, green for Released). Tapping navigates to FeedbackReviewScreen with the session ID.
Component Type	Flutter StatelessWidget

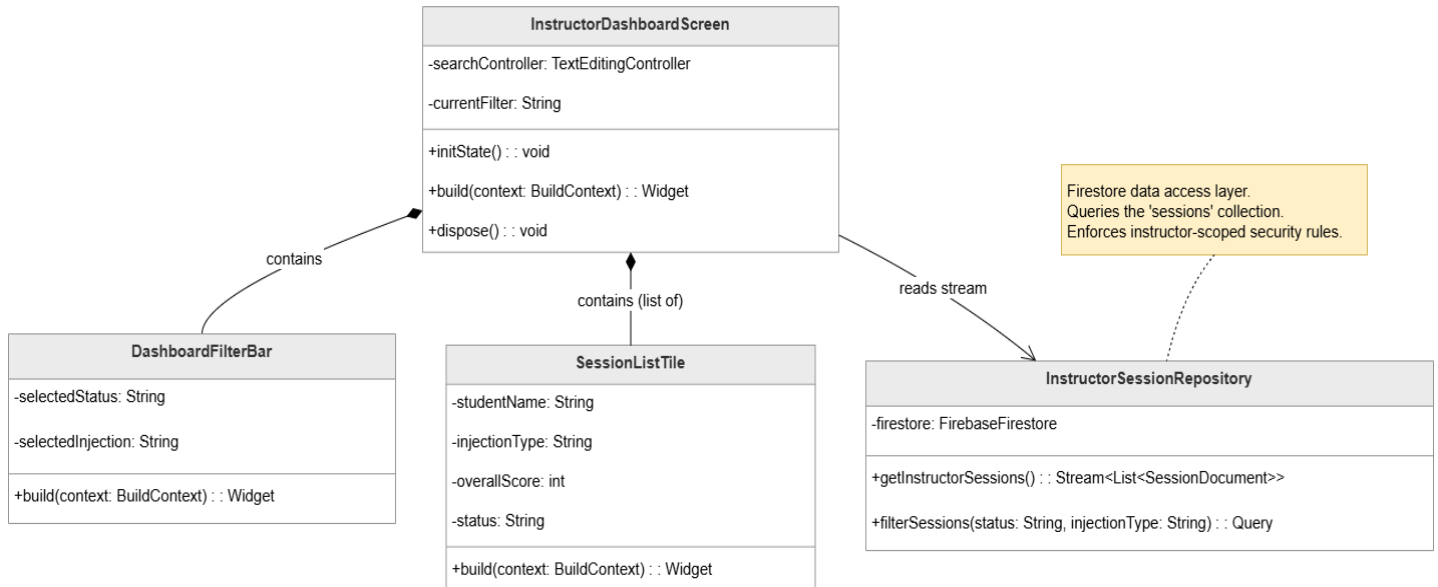
Component Name	DashboardFilterBar (Flutter StatelessWidget)
Description	Horizontal scrollable row of FilterChip widgets for filtering the session list by status (Pending / Released) and injection type (All / IM / SubQ / IV / ID).
Component Type	Flutter StatelessWidget

Back-end Components

Component Name	InstructorSessionRepository (Dart class)
Description	Firestore data access layer for instructor-scoped session records. Provides getInstructorSessions() returning a real-time Stream of all session documents, and filterSessions(status, injectionType) for filtered queries. Enforces instructor-scoped Firestore security rules server-side.
Component Type	Dart Repository Class

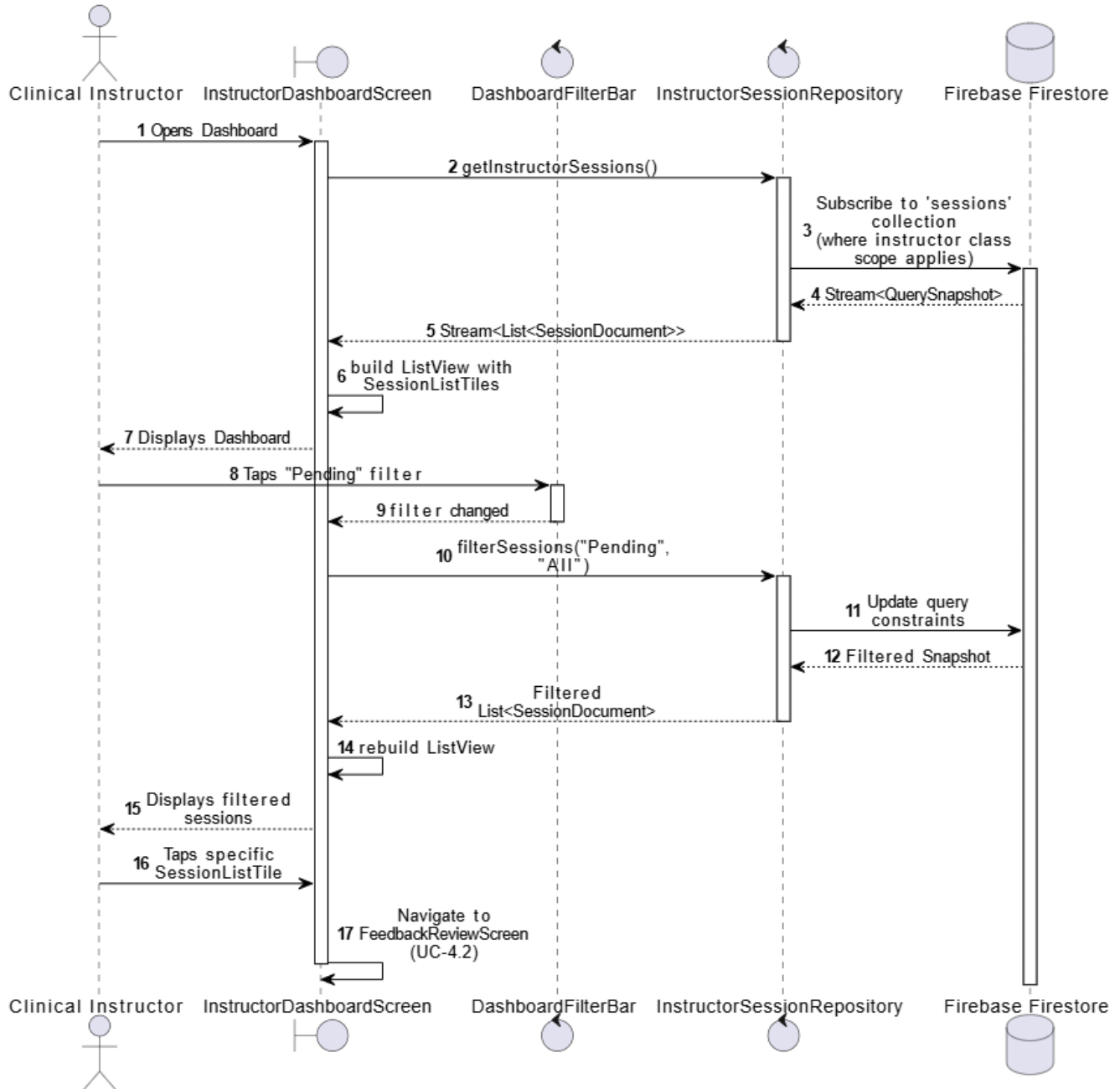
Object-Oriented Components Class Diagram

Class Diagram — UC-4.1: Instructor Performance Monitoring Dashboard



Sequence Diagram

Sequence Diagram — UC-4.1: Instructor Performance Monitoring Dashboard



Data Design

The dashboard reads from the Firestore 'sessions' collection. Firestore security rules enforce that instructors can only read and write session documents within their authorized class scope. No additional schema beyond the session document defined in Module 2.

4.2 AI Feedback Review and Release (UC-4.2)

User Interface Design

The Feedback Review screen shows the student's name, injection type, session date, and score at the top. Below are the three detection result cards (Insertion Angle, Aspiration, Withdrawal Angle with scores). The AI Generated Feedback section shows the Llama 3.3 draft text with an editable text area the instructor can modify. Two action buttons are shown: Release Feedback to Student (navy filled) and Edit Feedback First (outlined). A confirmation dialog appears before release. A Release Timestamp is recorded in Firestore on confirmation.

Front-end Components

Component Name	FeedbackReviewScreen (Flutter StatefulWidget)
Description	Instructor feedback review screen. Displays detection results read-only and the AI feedback text in an editable TextFormField. Manages local edit state and shows a confirmation AlertDialog before calling the release action. Handles Firestore update failure with an error snackbar.
Component Type	Flutter StatefulWidget / Screen

Component Name	FeedbackEditorWidget
Description	Editable text area widget pre-populated with the Llama 3.3 generated feedback text. Allows the instructor to modify, annotate, or replace the AI-generated content before release. Shows a character count and an unsaved changes indicator.
Component Type	Flutter StatefulWidget

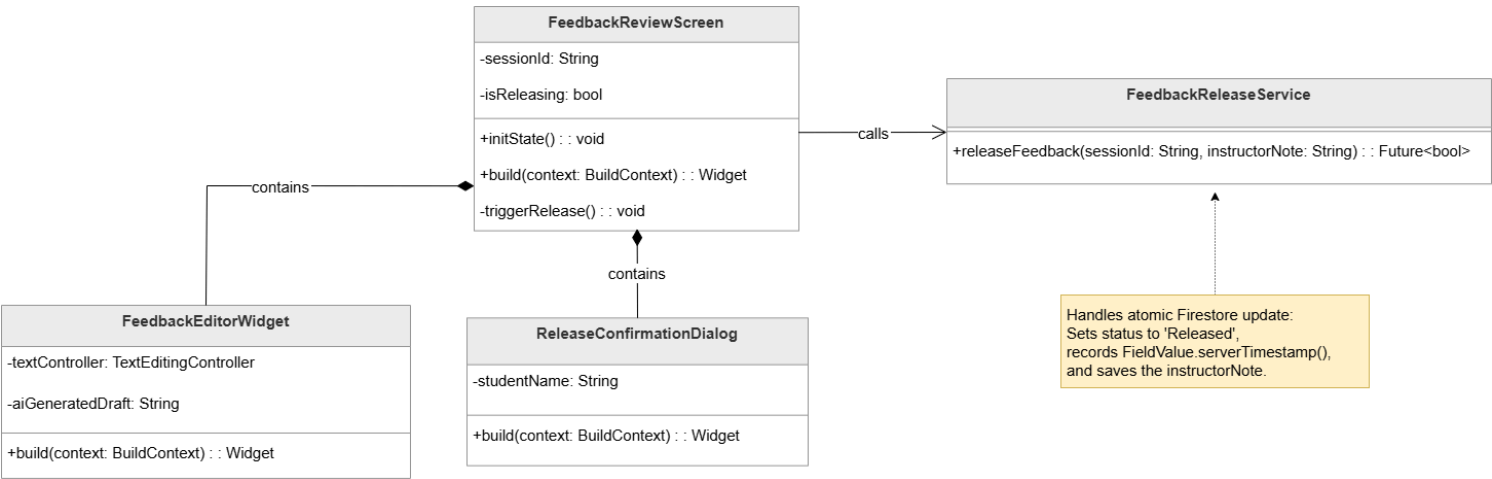
Component Name	ReleaseConfirmationDialog
Description	AlertDialog widget prompting the instructor to confirm feedback release. Shows the student's name and a summary of the action. Contains Cancel and Confirm Release buttons.
Component Type	Flutter StatelessWidget

Back-end Components

Component Name	FeedbackReleaseService (Dart class)
Description	Handles the Firestore atomic update on feedback release: sets feedbackStatus to Released, records the releaseTimestamp (server-side FieldValue.serverTimestamp()), and saves any instructor-edited feedback text to the instructorNote field. Implements one retry on Firestore update failure. Returns a typed result (success / failure) to the UI.
Component Type	Dart Service Class

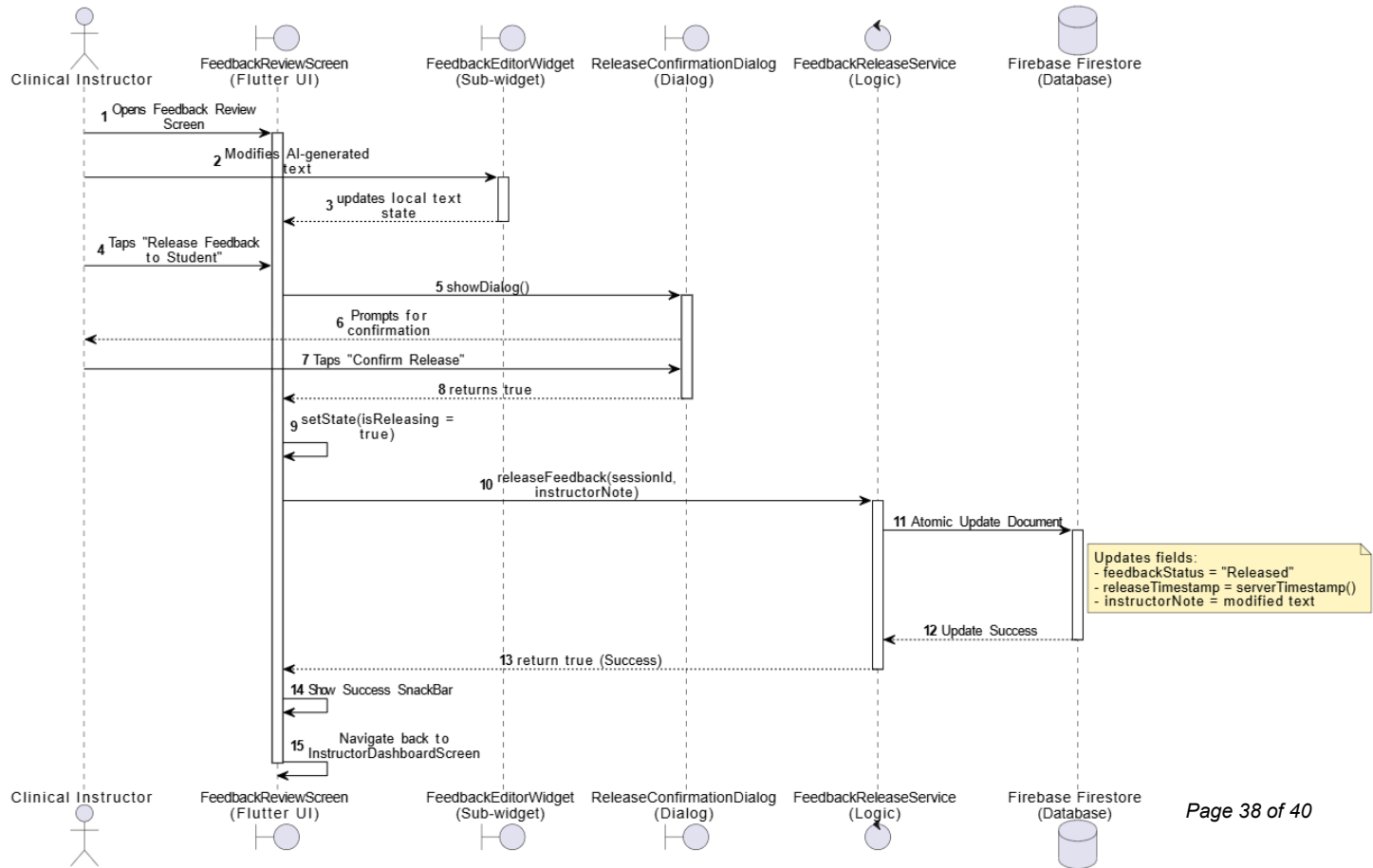
Object-Oriented Components
Class Diagram

Class Diagram — UC-4.2: Feedback Review and Release



Sequence Diagram

Sequence Diagram — UC-4.2: AI Feedback Review and Release



Data Design

Fields updated in the Firestore session document upon feedback release:

Field	Type	Description
feedbackStatus	String	Updated from Pending to Released on instructor confirmation
releaseTimestamp	Timestamp	Server-side timestamp recorded at moment of release
instructorNote	String	Instructor-edited or annotated feedback text (may differ from aiFeedbackText)
aiFeedbackText	String	Original Llama 3.3 generated text (preserved, not overwritten)

4. Overall Data Design — Firestore Schema

PRISM uses two Firestore collections:

Collection: users

Field	Type	Notes
uid (doc ID)	String	Firebase Auth UID
fullName	String	
email	String	
role	String	Student or Instructor
createdAt	Timestamp	Server timestamp
emailVerified	bool	

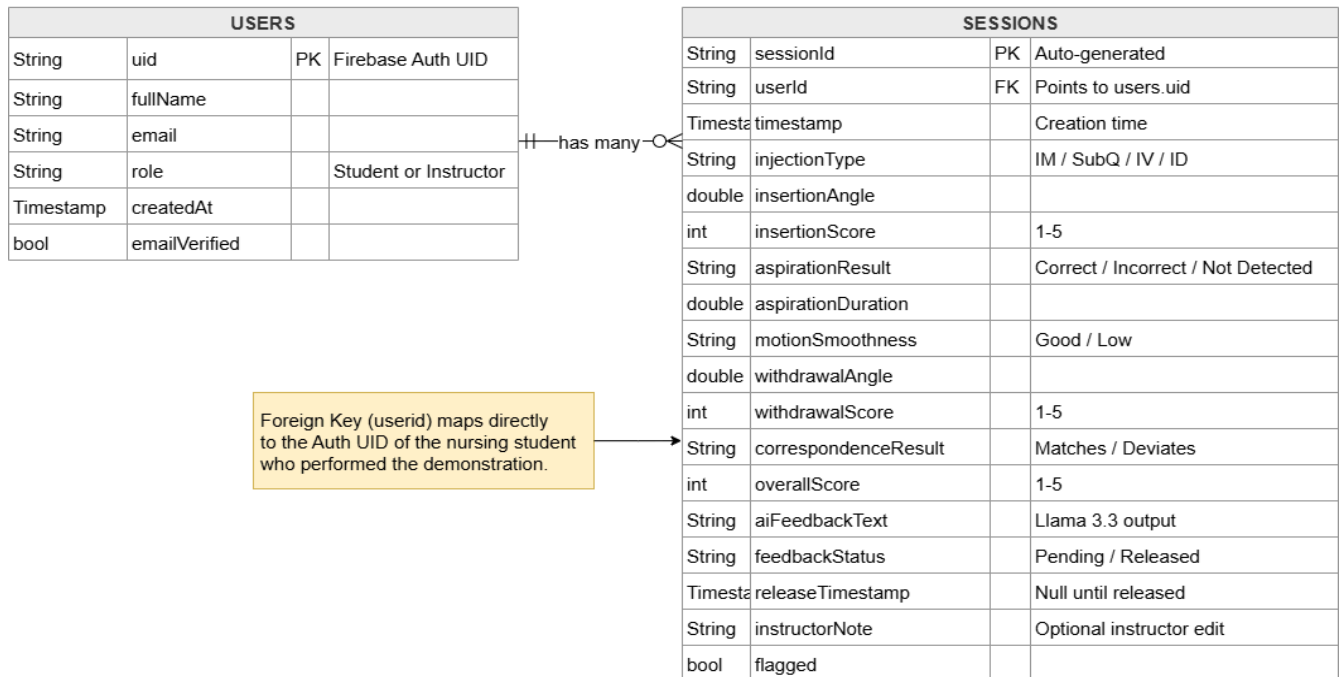
Collection: sessions

Field	Type	Notes
sessionId (doc ID)	String	Auto-generated
userId	String	Foreign key → users.uid
timestamp	Timestamp	Session creation time
injectionType	String	IM / SubQ / IV / ID
insertionAngle	double	Degrees
insertionScore	int	1–5

aspirationResult	String	Correct / Incorrect / Not Detected
aspirationDuration	double	Seconds
motionSmoothness	String	Good / Low
withdrawalAngle	double	Degrees
withdrawalScore	int	1–5
correspondenceResult	String	Matches / Deviates
overallScore	int	1–5
aiFeedbackText	String	Llama 3.3 output
feedbackStatus	String	Pending / Released / Feedback Generation Failed
releaseTimestamp	Timestamp	Null until released
instructorNote	String	Optional instructor edit
flagged	bool	True if any component Not Detectable

Entity Relationship Diagram

Entity Relationship Diagram — PRISM Firestore Collections



Relationship: One-to-Many (1:N) mapping from the **users** collection to individual **sessions** records.